

Архитектура и основные классы системы

Общая идея архитектуры

Архитектура игры построена по принципу разделения ответственности между игровыми системами. Каждая подсистема отвечает только за свою часть логики: управление игрой, персонажи, бой, предметы, инвентарь, диалоги, локации, торговлю и загрузку данных.

Главным управляющим классом является Game. Он не реализует все игровые механики самостоятельно, а координирует работу остальных подсистем. Это позволяет не смешивать боевую систему, диалоги, торговлю, инвентарь и загрузку данных в одном месте.

Основные подсистемы игры:

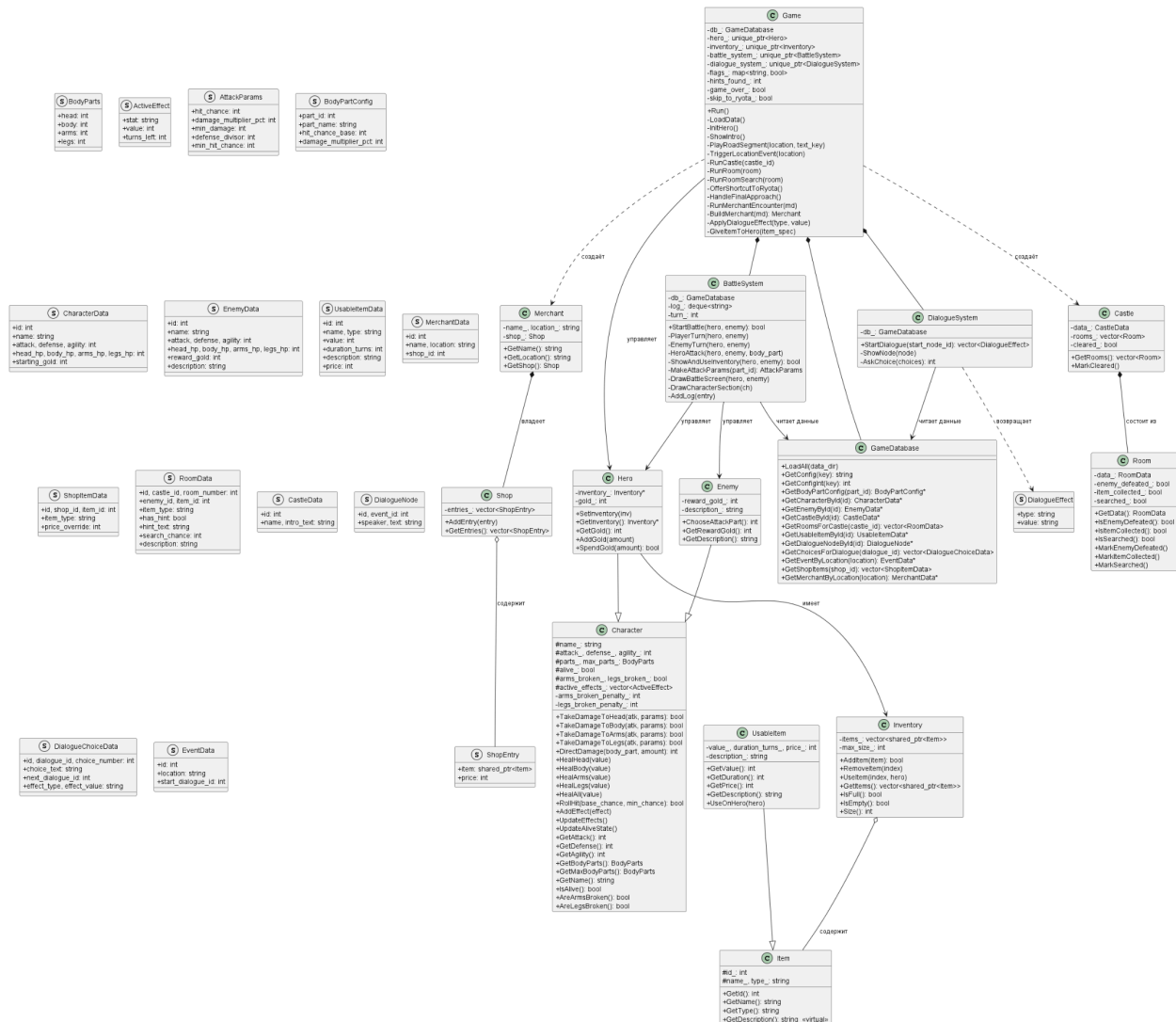
- Game — управление игровым процессом;
- GameDatabase — загрузка и хранение игровых данных;
- Character, Hero, Enemy — система персонажей;
- BattleSystem — пошаговая боевая система;
- Item, UsableItem, Inventory — предметы и инвентарь;
- Castle, Room — система локаций и комнат;
- DialogueSystem — система диалогов;
- Merchant, Shop — система торговли.

Архитектура проекта построена с использованием data-driven подхода. Большая часть игрового контента хранится в CSV-файлах и загружается через GameDatabase.

На общей UML-схеме классов показано, что Game является центральным управляющим классом.

Он:

- запускает бой через BattleSystem;
- запускает диалоги через DialogueSystem;
- управляет прохождением локаций через Castle и Room;
- взаимодействует с торговлей через Merchant и Shop;
- загружает игровые данные через GameDatabase;
- управляет героем, инвентарём и игровыми событиями.



На общей UML-схеме классов показано, что Game является центральным управляющим классом.

Он:

- запускает бой через BattleSystem;
- запускает диалоги через DialogueSystem;
- управляет прохождением локаций через Castle и Room;
- взаимодействует с торговлей через Merchant и Shop;
- загружает игровые данные через GameDatabase;
- управляет героем, инвентарём и игровыми событиями.

GameDatabase используется как единый источник игровых данных для остальных подсистем.

BattleSystem отвечает только за боевую логику.

DialogueSystem отвечает только за выполнение диалогов.

Inventory управляет хранением и использованием предметов.
Castle и Room отвечают за структуру игровых локаций.

Такое разделение делает архитектуру более понятной, уменьшает связность между системами и упрощает дальнейшее развитие проекта.

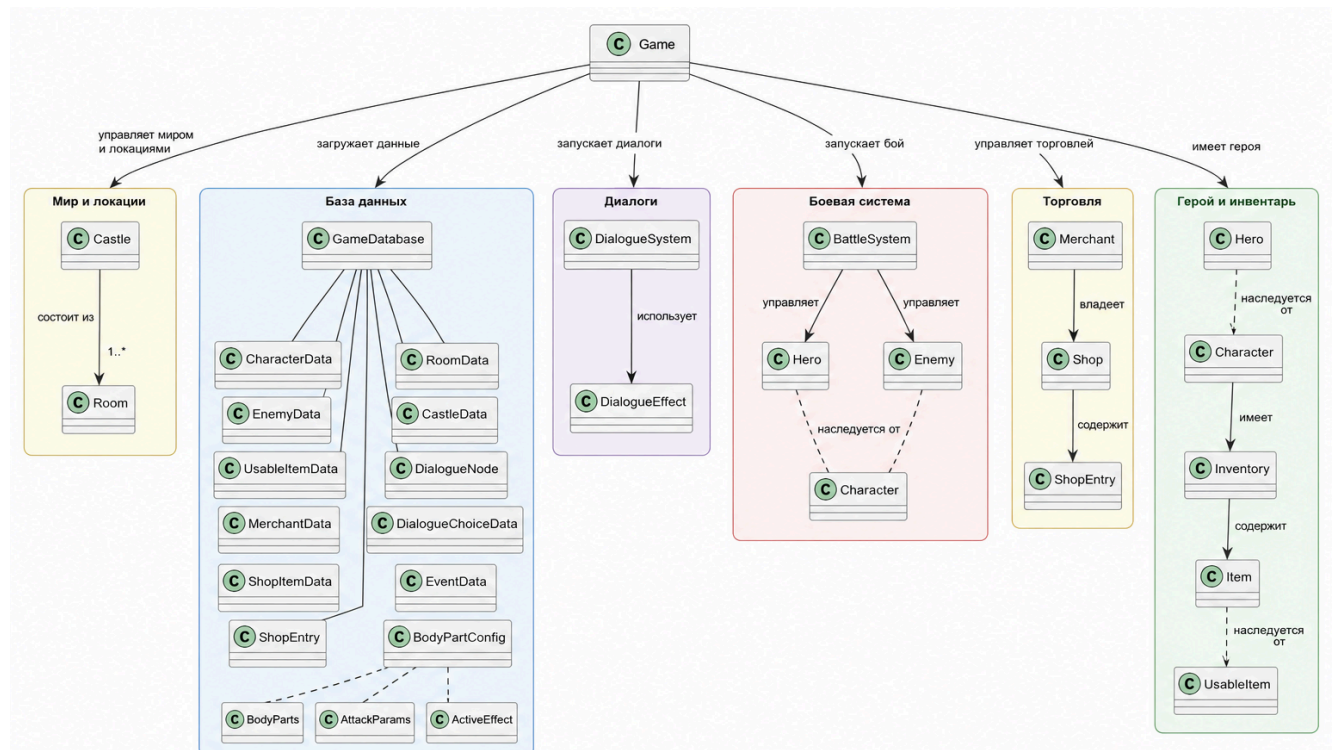
Диаграмма классов

Для описания структуры проекта используется UML-диаграмма классов. На ней показаны основные игровые подсистемы, связи между классами и распределение ответственности между ними.

Класс Character является базовым для Hero и Enemy. В нём реализована общая логика персонажей: характеристики, части тела, получение урона, лечение и временные эффекты.

Класс Item является базовым для UsableItem. Используемые предметы могут лечить части тела, накладывать эффекты или наносить прямой урон.

Класс GameDatabase хранит игровые данные: врагов, предметы, комнаты, замки, диалоги и параметры частей тела. Эти данные используются при создании игровых объектов и загрузке игрового контента.



Класс Game

 Game
-db_: GameDatabase -hero_: unique_ptr<Hero> -inventory_: unique_ptr<Inventory> -battle_system_: unique_ptr<BattleSystem> -dialogue_system_: unique_ptr<DialogueSystem> -flags_: map<string, bool> -hints_found_: int -game_over_: bool -skip_to_ryota_: bool
+Run() -LoadData() -InitHero() -ShowIntro() -PlayRoadSegment(location, text_key) -TriggerLocationEvent(location) -RunCastle(castle_id) -RunRoom(room) -RunRoomSearch(room) -OfferShortcutToRyota() -HandleFinalApproach() -RunMerchantEncounter(md) -BuildMerchant(md): Merchant -ApplyDialogueEffect(type, value) -GiveItemToHero(item_spec)

Назначение

Класс Game является главным управляющим классом игры. Он отвечает за запуск проекта, основной игровой цикл и переключение между игровыми этапами.

Отвечает за

- запуск игры;
- инициализацию основных систем;
- основной игровой цикл;
- переходы между локациями, боями, диалогами и торговлей;
- завершение игры.

Не отвечает за

- расчёт урона;
- хранение характеристик персонажей;
- конкретную логику диалогов;
- применение предметов;
- загрузку таблиц напрямую.

Связи

Game использует BattleSystem, DialogueSystem, GameDatabase, Castle и Merchant.

Класс GameDatabase

 GameDatabase
□ weapons: vector<WeaponData> □ usableItems: vector<UsableItemData> □ castles: vector<CastleData> □ enemies: vector<EnemyData> □ rooms: vector<RoomData> □ dialogues: vector<DialogueChoiceData>
● loadWeapons() ● loadUsableItems() ● loadEnemies() ● loadRooms() ● loadDialogues()

Назначение

Класс GameDatabase отвечает за загрузку игровых данных из CSV-файлов и хранение этих данных в памяти.

Отвечает за:

- загрузку врагов;
- загрузку используемых предметов;
- загрузку комнат;
- загрузку замков;
- загрузку диалогов;
- загрузку конфигурации частей тела.

Не отвечает за

- проведение боя;
- поведение персонажей;
- применение предметов;
- управление игровым процессом.

Основные структуры данных:

- CharacterData;
- EnemyData;
- UsableItemData;
- RoomData;
- CastleData;
- DialogueChoiceData;
- BodyPartConfig.

GameDatabase реализует data-driven подход: игровые параметры отделены от кода и могут изменяться без перекомпиляции проекта.

Принцип работы

Данные игры могут быть описаны в таблицах. Например, в таблице оружия указывается название, тип, урон и точность. В таблице врагов указываются характеристики и здоровье частей тела. В таблице комнат указывается, к какому замку относится комната, какой враг находится внутри и какой предмет можно получить.

GameDatabase загружает эти данные, а игровые классы используют их для создания объектов. Таким образом, данные отделены от игровой логики.

Класс Character



Назначение

Класс **Character** является базовым классом для всех персонажей игры. Он хранит характеристики персонажа, состояние частей тела, временные эффекты, а также реализует получение урона и лечение.

Отвечает за

- имя персонажа;
- базовые характеристики: атака, защита, ловкость;
- здоровье частей тела;
- проверку попадания;

- получение урона;
- лечение частей тела;
- временные эффекты;
- проверку состояния жизни.

Не отвечает за

- проведение боя целиком;
- выбор действия игроком;
- управление игрой;
- диалоги;
- торговлю.

Особенность системы здоровья

В игре нет одного общего HP. Здоровье разделено по частям тела:

- голова;
- тело;
- руки;
- ноги.

Голова и тело являются критическими зонами. Если здоровье головы или тела становится равным нулю, персонаж погибает. Если здоровье рук становится равным нулю, персонаж получает штраф к атаке. Если здоровье ног становится равным нулю, персонаж получает штраф к ловкости. Эти штрафы действуют до лечения соответствующей части тела.

Класс Hero



Назначение

Класс Hero описывает главного героя игры и наследуется от Character.

Отвечает за

- действия героя в бою;
- атаки по частям тела;
- связь с инвентарём;
- использование предметов через Inventory.

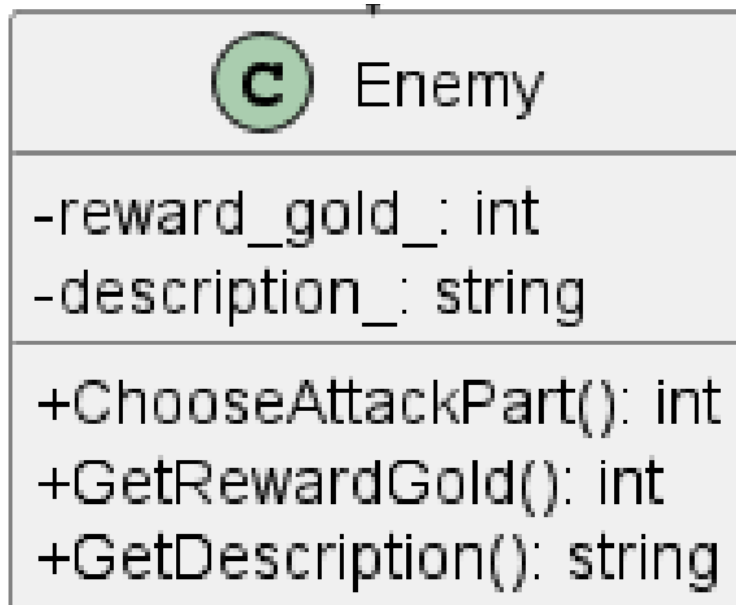
Не отвечает за

- проведение боя целиком;
- управление всей игрой;
- поведение врагов;
- диалоги;
- торговлю.

Связи

Hero наследуется от Character и имеет связь с Inventory.

Класс Enemy



Назначение

Класс Enemy описывает врага и наследуется от Character.

Отвечает за

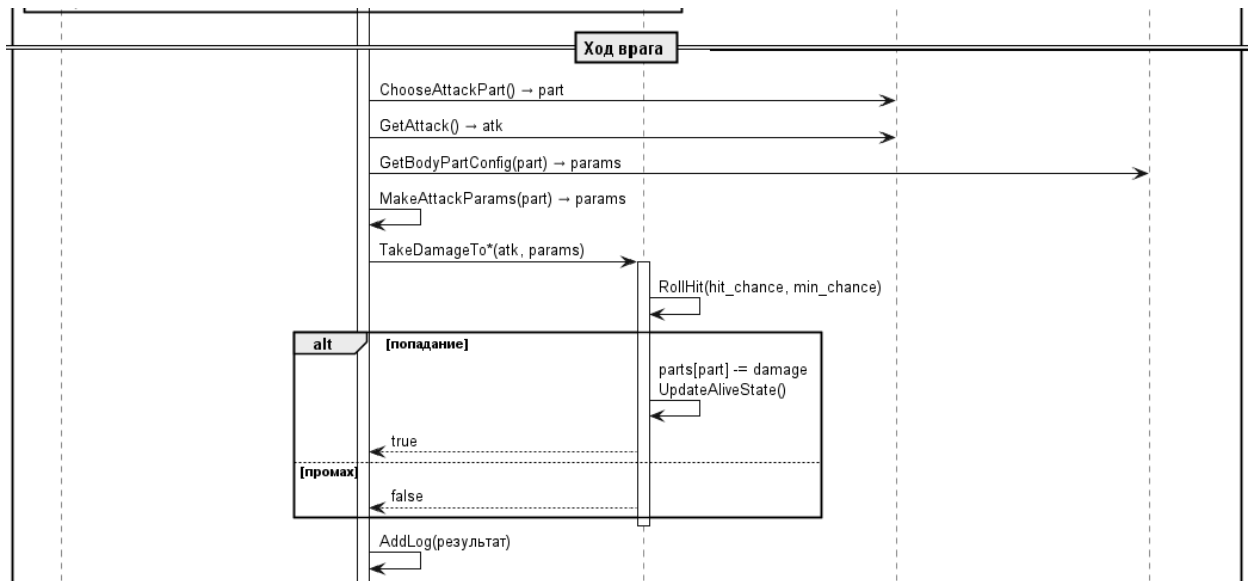
- характеристики врага;
- состояние врага;
- случайный выбор части тела для атаки;
- участие в бою.

Не отвечает за

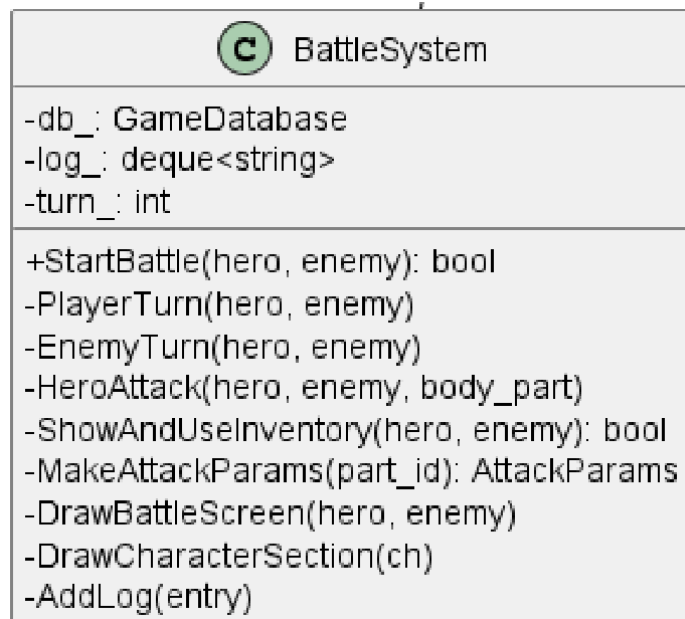
- управление боем целиком;
- управление игрой;
- диалоги;
- торговлю.

Особенность

Враг выбирает часть тела для атаки случайно. Это делает бой менее предсказуемым и не требует отдельной сложной системы искусственного интеллекта.



Класс BattleSystem



Назначение

Класс BattleSystem отвечает за проведение пошагового боя между героем и врагом.

Отвечает за

- запуск боя;
- порядок ходов;
- ход игрока;
- ход врага;
- проверку победы или поражения;
- обновление временных эффектов после ходов.

Не отвечает за

- хранение характеристик персонажей;
- хранение предметов;
- загрузку данных;
- диалоги;
- торговлю.

Принцип работы боя

Бой проходит пошагово. Сначала игрок выбирает часть тела врага для атаки. После этого система проверяет попадание и рассчитывает урон. Если враг выжил, он атакует случайную часть тела героя. После ходов обновляются временные эффекты обоих персонажей.

Бой завершается, когда герой или враг погибает.

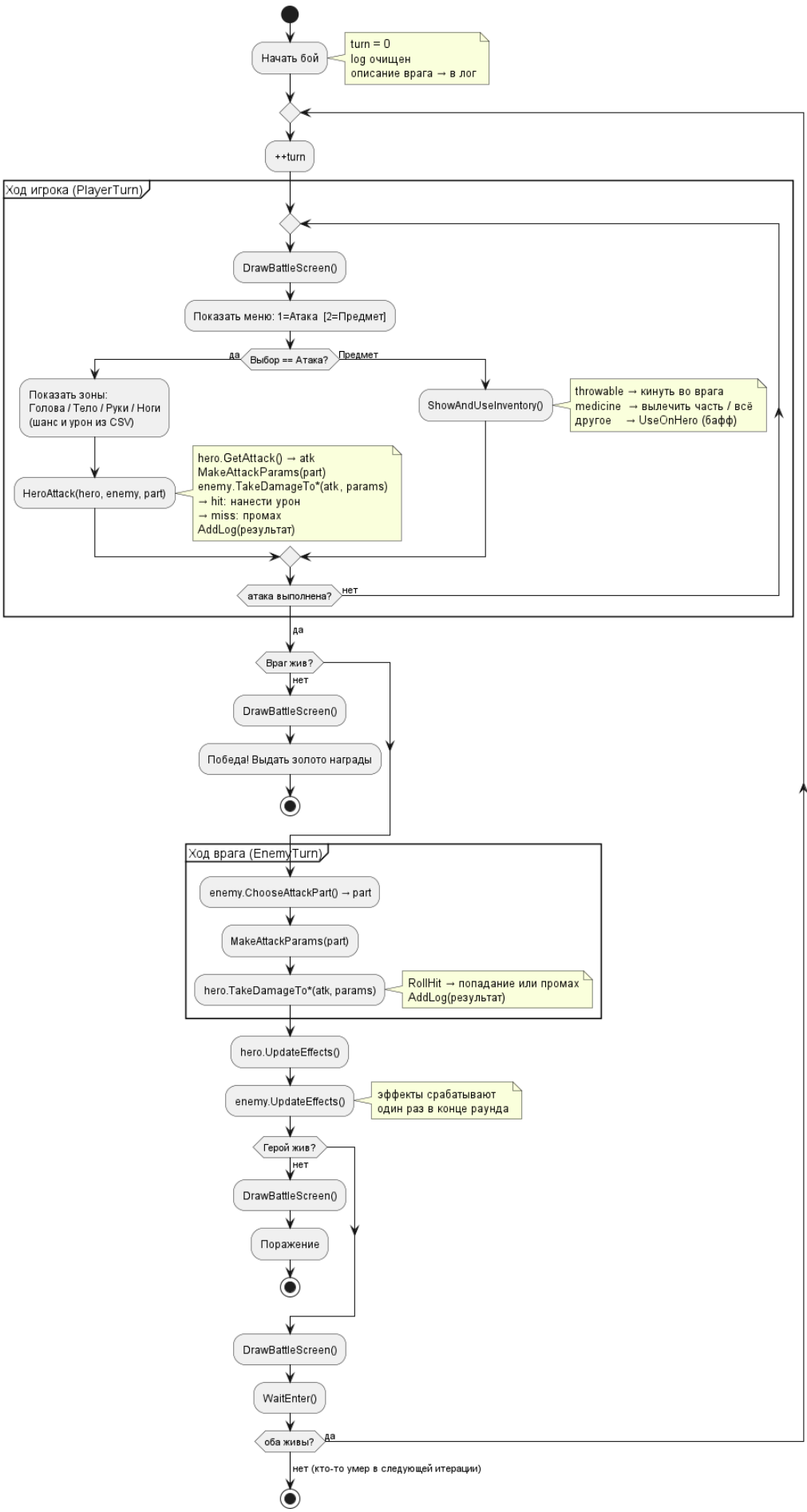
Система боевого лога

BattleSystem ведёт текстовый журнал боя.

В лог записываются:

- попадания;
- промахи;
- нанесённый урон;
- использование предметов;
- лечение;
- применение эффектов;
- победа и поражение.

Боевой лог нужен для отображения результатов действий игрока и врага во время боя.



Вероятности и формулы боевой системы

В боевой системе используется случайная проверка попадания. Для проверки генерируется случайное число от 1 до 100.

Итоговый шанс попадания рассчитывается по формуле:

$$\text{finalChance} = \text{baseChance} - \text{agility}$$

где:

- `baseChance` — базовый шанс попадания по выбранной части тела;
- `agility` — ловкость цели.

Если итоговый шанс попадания становится меньше 10, он приравнивается к 10:

$$\text{if } \text{finalChance} < 10, \text{ then } \text{finalChance} = 10$$

Атака считается успешной, если:

$$\text{randValue} < \text{finalChance}$$

Итоговый урон рассчитывается по формуле:

$$\text{damage} = \text{attack} * \text{multiplier} - \text{defense}$$

где:

- `attack` — атака нападающего;
- `multiplier` — множитель выбранной части тела;
- `defense` — защита цели.

Если итоговый урон меньше 1, он приравнивается к 1:

$$\text{if } \text{damage} < 1, \text{ then } \text{damage} = 1$$

Голова

$$\text{baseChance} = 20$$

$$\text{multiplier} = 2.0$$

$$\text{damage} = \text{attack} * 2 - \text{defense}$$

Особенности:

- самый высокий урон;
- самый низкий шанс попадания;
- при снижении здоровья головы до 0 персонаж погибает.

Тело

baseChance = 80

multiplier = 1.0

damage = attack - defense

Особенности:

- стабильный урон;
- высокий шанс попадания;
- при снижении здоровья тела до 0 персонаж погибает.

Руки

baseChance = 65

multiplier = 0.9

damage = attack * 0.9 - defense

Особенности:

- урон ниже обычного;
- при снижении здоровья рук до 0 персонаж получает штраф к атаке;
- штраф действует до лечения рук.

Ноги

baseChance = 60

multiplier = 0.8

damage = attack * 0.8 - defense

Особенности:

- урон ниже обычного;
 - при снижении здоровья ног до 0 персонаж получает штраф к ловкости;
 - штраф действует до лечения ног.
-

Система временных эффектов

В игре есть временные эффекты: бафы и дебафы. Они не изменяют базовые характеристики навсегда, а действуют ограниченное количество ходов.

Каждый временный эффект содержит:

- характеристику, которую он изменяет;
- величину изменения;
- количество оставшихся ходов.

Примеры эффектов:

- дымовая бомба временно увеличивает ловкость героя;
- предмет усиления атаки временно повышает атаку;
- предмет защиты временно повышает защиту.

После каждого хода количество оставшихся ходов у эффекта уменьшается. Когда оно становится равно нулю, эффект перестаёт учитываться.

Система лечения

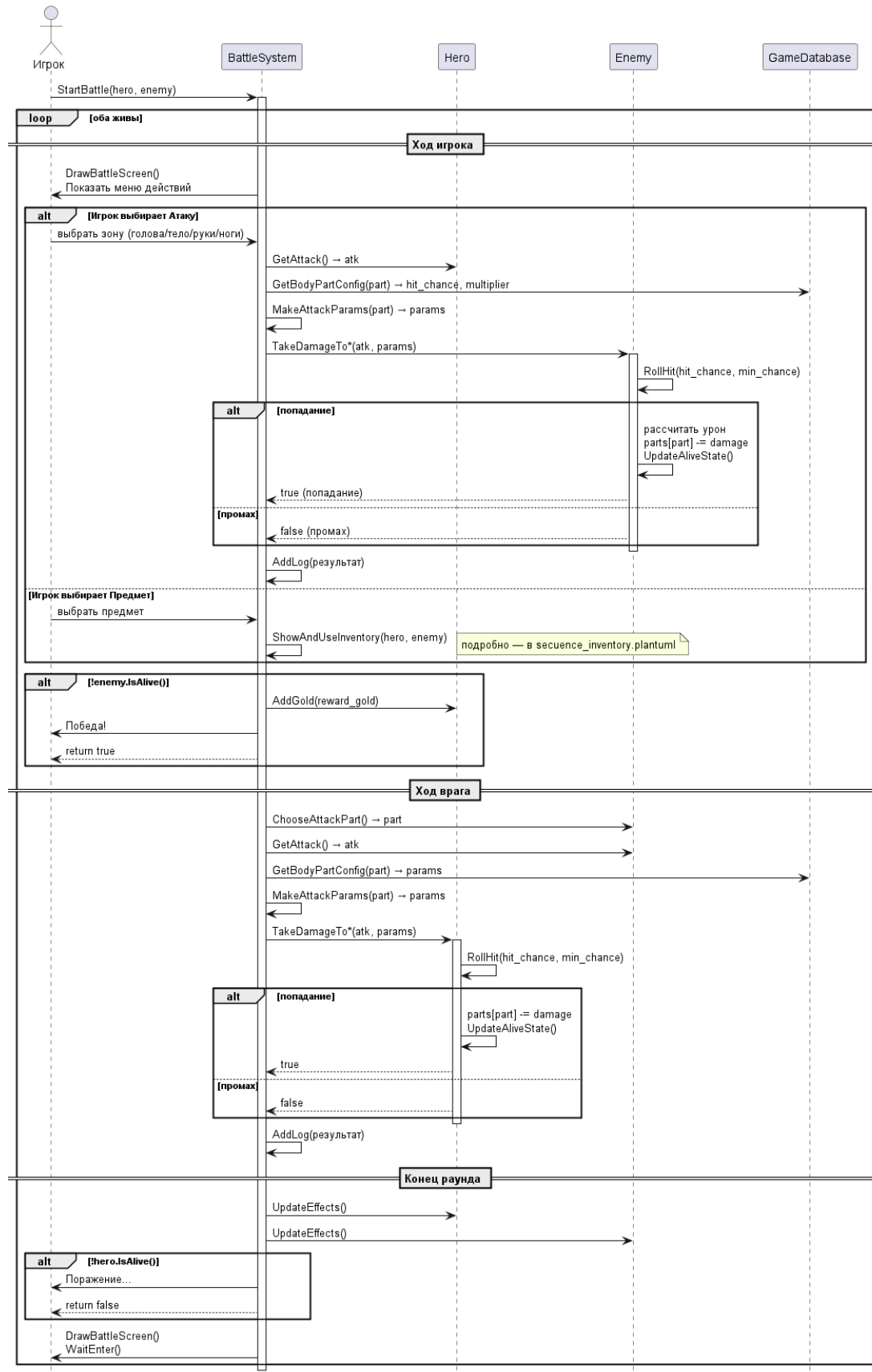
Лечение восстанавливает здоровье частей тела. Используемые предметы могут лечить:

- голову;
- тело;
- руки;
- ноги;
- все части тела сразу.

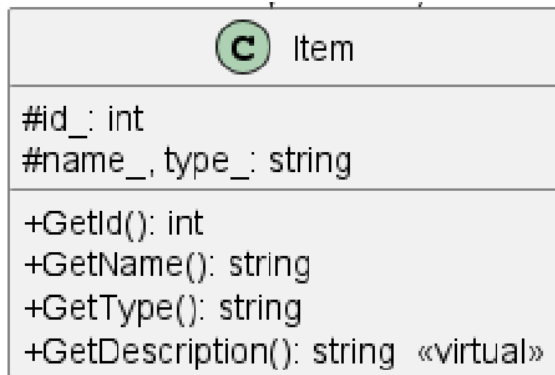
Если после лечения здоровье рук становится больше нуля, штраф к атаке снимается.

Если после лечения здоровье ног становится больше нуля, штраф к ловкости снимается.

Лечение головы и тела может восстановить здоровье, но если персонаж уже погиб, лечение не используется как воскрешение. Это значит, что смерть остаётся финальным состоянием боя.



Класс Item



Назначение

Класс Item является базовым классом для всех предметов игры.

Отвечает за

- идентификатор предмета;
- название предмета;
- тип предмета;
- общую основу для наследников.

Не отвечает за

- хранение предметов;
- продажу предметов;
- конкретное применение предмета;
- управление игрой.

Класс UsableItem

 UsableItem
-value_: int -duration_turns_: int -price_: int -description_: string
+GetValue(): int +GetDuration(): int +GetPrice(): int +GetDescription(): string +UseOnHero(hero)

Назначение

Класс UsableItem описывает используемые предметы.

Отвечает за

- лечение частей тела;
- временное усиление характеристик;
- создание временных эффектов;
- применение предмета к герою.
- прямой урон метательных предметов.

Не отвечает за

- хранение предметов;
- продажу предметов;
- управление игрой.

Примеры

Медицина относится к UsableItem, потому что восстанавливает здоровье частей тела.
Дымовая бомба также относится к UsableItem, потому что временно повышает ловкость героя и снижает шанс попадания по нему.

Класс Inventory

 Inventory
<pre>-items_: vector<shared_ptr<Item>> -max_size_: int</pre>
<pre>+AddItem(item): bool +RemoveItem(index) +UseItem(index, hero) +GetItems(): vector<shared_ptr<Item>> +IsFull(): bool +IsEmpty(): bool +Size(): int</pre>

Назначение

Класс Inventory хранит предметы героя и позволяет использовать выбранный предмет.

Инвентарь хранит список объектов базового типа Item. В текущей версии игры все предметы являются наследниками UsableItem.

При использовании предмета система определяет его реальный тип и применяет соответствующую игровую логику:

Отвечает за

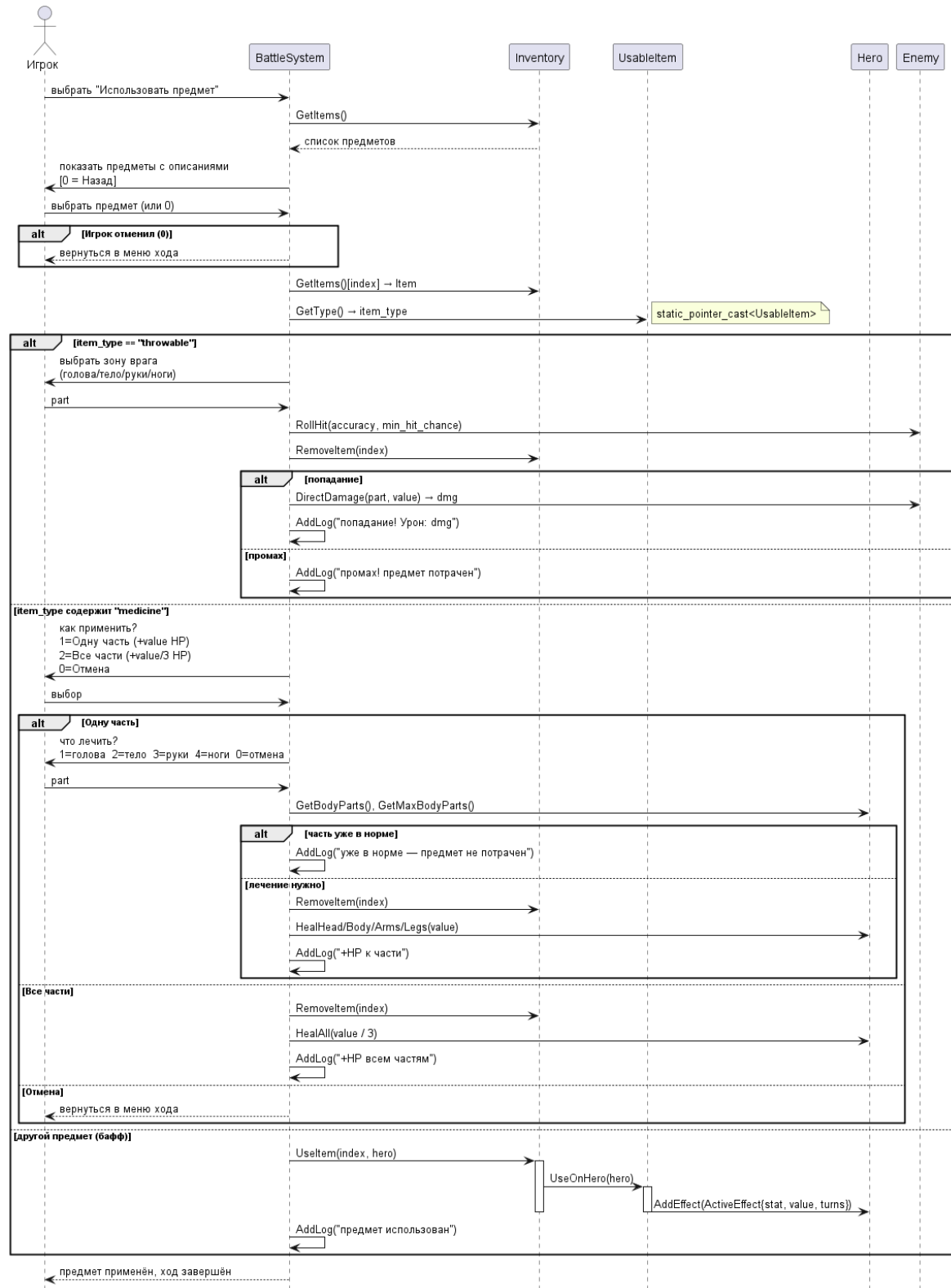
- хранение предметов;
- добавление предметов;
- удаление предметов;
- использование предметов.

Не отвечает за

- расчёт урона;
- проведение боя целиком;
- продажу предметов;
- загрузку данных.

Принцип работы

Инвентарь хранит список предметов типа Item. При использовании предмета система определяет его реальный тип:



Класс Room

 Room
-data_: RoomData -enemy_defeated_: bool -item_collected_: bool -searched_: bool
+GetData(): RoomData +IsEnemyDefeated(): bool +IsItemCollected(): bool +IsSearched(): bool +MarkEnemyDefeated() +MarkItemCollected() +MarkSearched()

Назначение

Класс Room описывает одну комнату замка.

Отвечает за

- идентификатор комнаты;
- связь с замком;
- идентификатор врага;
- идентификатор предмета.

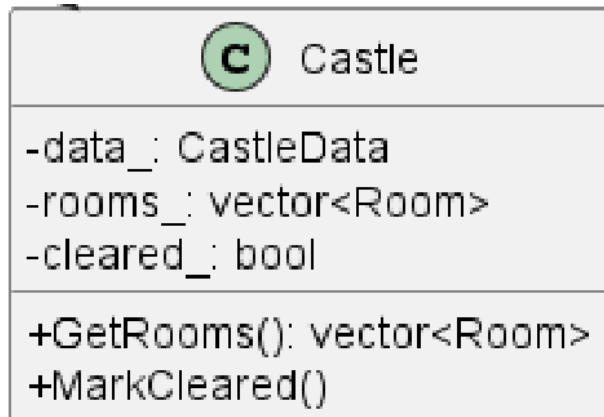
Не отвечает за

- проведение боя;
- управление игрой;
- диалоги;
- торговлю.

Особенность

Комната является минимальной единицей локации. Замок состоит из набора комнат.

Класс Castle



Назначение

Класс Castle описывает замок как набор комнат.

Отвечает за

- идентификатор замка;
- хранение комнат;
- порядок прохождения комнат;
- состояние прохождения замка.

Не отвечает за

- проведение боя;
- хранение характеристик врагов;
- торговлю;
- диалоги.

Принцип работы

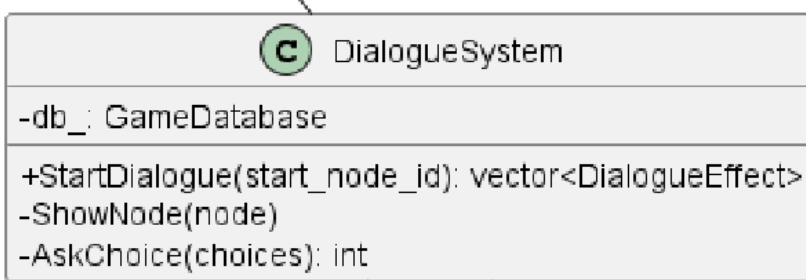
Замок формируется на основе данных комнат.

Каждая комната содержит идентификатор `castle_id`, благодаря которому игра определяет, к какому замку относится данная комната.

При создании замка `GameDatabase` загружает все комнаты с нужным `castle_id` и передаёт их в объект `Castle`.

Замок хранит комнаты в порядке прохождения и позволяет последовательно запускать их во время игры.

Класс DialogueSystem



Назначение

Класс `DialogueSystem` отвечает за выполнение диалогов.

Отвечает за

- запуск диалога;
- вывод текста;
- отображение вариантов ответа;
- получение выбора игрока;
- переход между шагами диалога;
- вызов callback-функции при выборе эффекта.

Не отвечает за

- проведение боя;
- торговлю;
- управление всей игрой;
- хранение предметов;
- изменение характеристик персонажа напрямую.

Принцип работы

Диалог работает как последовательность шагов.

На каждом шаге:

- выводится текст реплики;

- отображаются варианты ответа;
- игрок выбирает один из вариантов.

После выбора система:

- получает выбранный вариант;
- при необходимости вызывает callback;
- переходит к следующему шагу;
- либо завершает диалог.

Класс Shop

 Shop
-entries_: vector<ShopEntry>
+AddEntry(entry)
+GetEntries(): vector<ShopEntry>

Назначение

Класс Shop описывает магазин предметов.

Отвечает за

- хранение товаров;
- отображение ассортимента;
- продажу предметов;
- передачу купленного предмета в инвентарь.

Не отвечает за

- проведение боя;
- применение предметов;
- управление игрой;
- диалоги.

Класс Merchant



Назначение

Класс Merchant описывает странствующего торговца.

Отвечает за

- имя торговца;
- связь с магазином;
- открытие торговли с героем.

Не отвечает за

- проведение боя;
- прохождение замков;
- применение предметов;
- управление всей игрой.

Система торговли

Торговля реализована через классы Merchant и Shop.

Merchant описывает торговца и содержит магазин. Shop хранит список предметов, которые можно купить.

При покупке предмет добавляется в инвентарь героя. Сам магазин не применяет предметы и не участвует в бою, а только отвечает за отображение ассортимента и передачу купленного предмета игроку.